



**RV College of
Engineering®**

Go, change the world

Rashtreeya Sikshana Samithi Trust
RV College of Engineering®, Bengaluru.
(An Autonomous Institution Affiliated to VTU, Belagavi)

DEEP LEARNING

UNIT 2

Convolutional neural networks or CNNs, are a specialized kind of neural network for processing data that has a known grid-like topology.

Examples **include time-series data**, which can be thought of as a **1-D grid** taking samples at regular time intervals, and image data, which can be thought of as a 2-D grid of pixels.

The name “convolutional neural network” indicates that the **network employs a mathematical linear operation called convolution**. Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.

Convolutional networks stand out as an example of neuroscientific principles influencing deep learning.

The goal of this chapter is to describe the kinds of tools that convolutional networks provide

The Convolution Operation : In general form, convolution is an operation on two functions of a real-valued argument.

Suppose we are tracking the location of a spaceship with a laser sensor. Our laser sensor provides a single output $x(t)$, the position of the spaceship at time t . Both x and t are real valued, that is, we can get a different reading from the laser sensor at any instant in time.

Now suppose that our laser sensor is somewhat noisy. To obtain a less noisy estimate of the spaceship's position, we would like to average several measurements. Of course, more recent measurements are more relevant, so we will want this to be a weighted average that gives more weight to recent measurements.

We can do this with a weighting function $w(a)$, where a is the age of a measurement. If we apply such a weighted average operation at every moment, we obtain a new function s providing a smoothed estimate of the position of the spaceship:

$$s(t) = \int x(a)w(t-a)da.$$

$$s(t) = (x * w)(t).$$

This operation is called convolution . The convolution operation is typically denoted with an Asterisk:

In convolutional network terminology, the first argument (in this example, the function x) to the convolution is often referred to as the input , and the second argument (in this example, the function w) as **the kernel** . The output is sometimes referred to as **the feature map**.

when we work with data on a computer, time will be discretized, The time index t can then take on only integer values. If we now assume that x and w are defined only on integer t , we can define the discrete convolution:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a).$$

if we use a two-dimensional image I as our input, we probably also want to use a two-dimensional kernel K

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i-m, j-n).$$

Convolution is commutative, meaning we can equivalently write

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i-m, j-n)K(m, n).$$

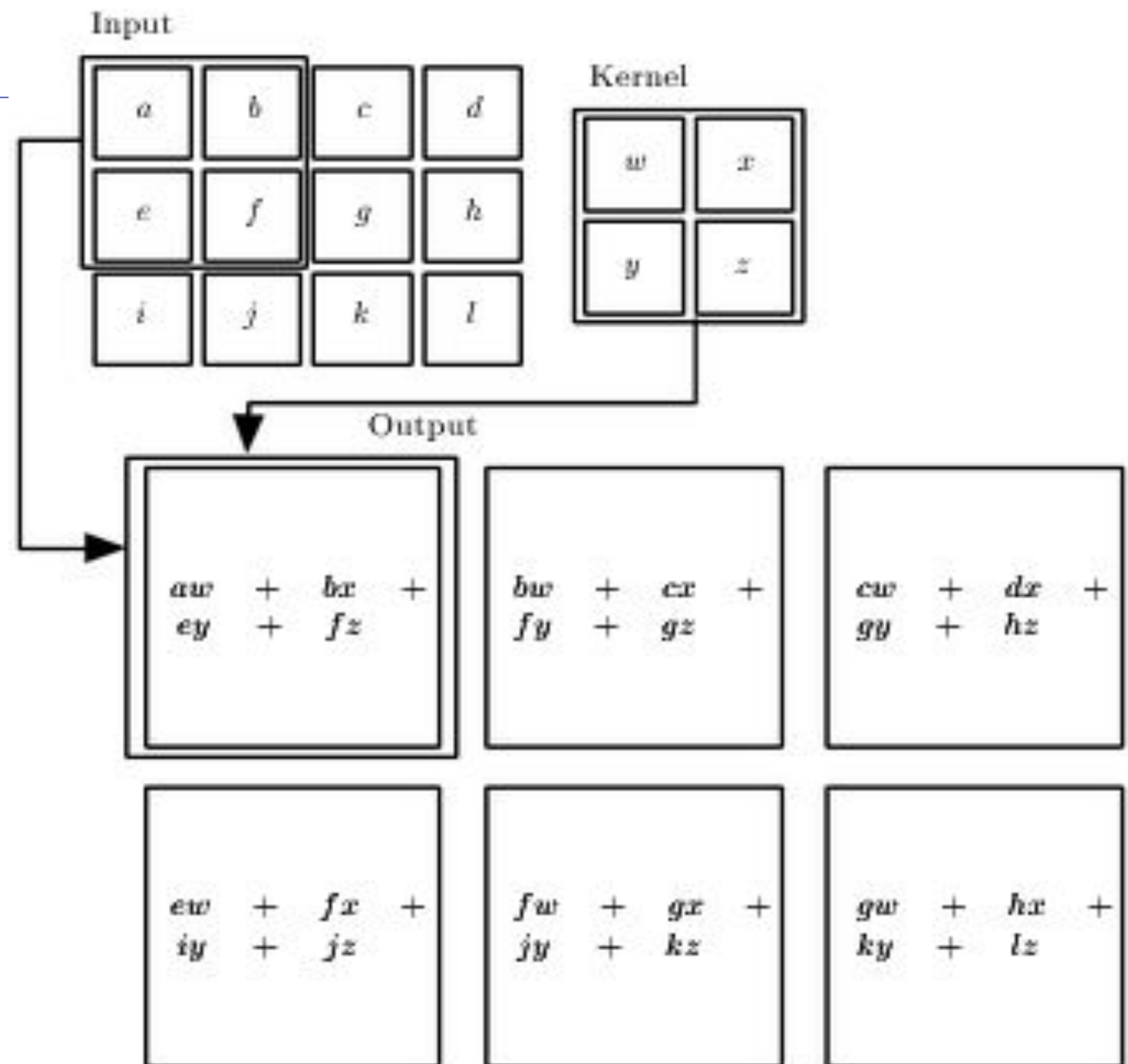
The commutative property of convolution arises because we **have flipped** the kernel relative to the input, in the sense that as m increases, the index into the input increases, but the index into the kernel decreases.

The only reason to flip the kernel is to obtain the commutative property. While the commutative property is useful for writing proofs, it is not usually an important property of a neural network implementation. Instead, many neural network libraries implement a related function called **the cross-correlation**, which is the same as convolution but without flipping the kernel.

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Discrete convolution can be viewed as multiplication by a matrix, but the matrix has several entries constrained to be equal to other entries. For example, for univariate discrete convolution, each row of the matrix is constrained to be equal to the row above shifted by one element. This is known as a **Toeplitz matrix**. In two dimensions, a doubly block **circulant matrix** corresponds to convolution

Figure 9.1: An example of 2-D convolution without kernel flipping. We restrict the output to only positions where the kernel lies entirely within the image, called “valid” convolution in some contexts. We draw boxes with arrows to indicate how the upper-left element of the output tensor is formed by applying the kernel to the corresponding upper-left region of the input tensor



Convolution leverages three important ideas that can help improve a machine learning system: sparse interactions (also referred to as sparse connectivity or sparse weights), parameter sharing and equivariant representations

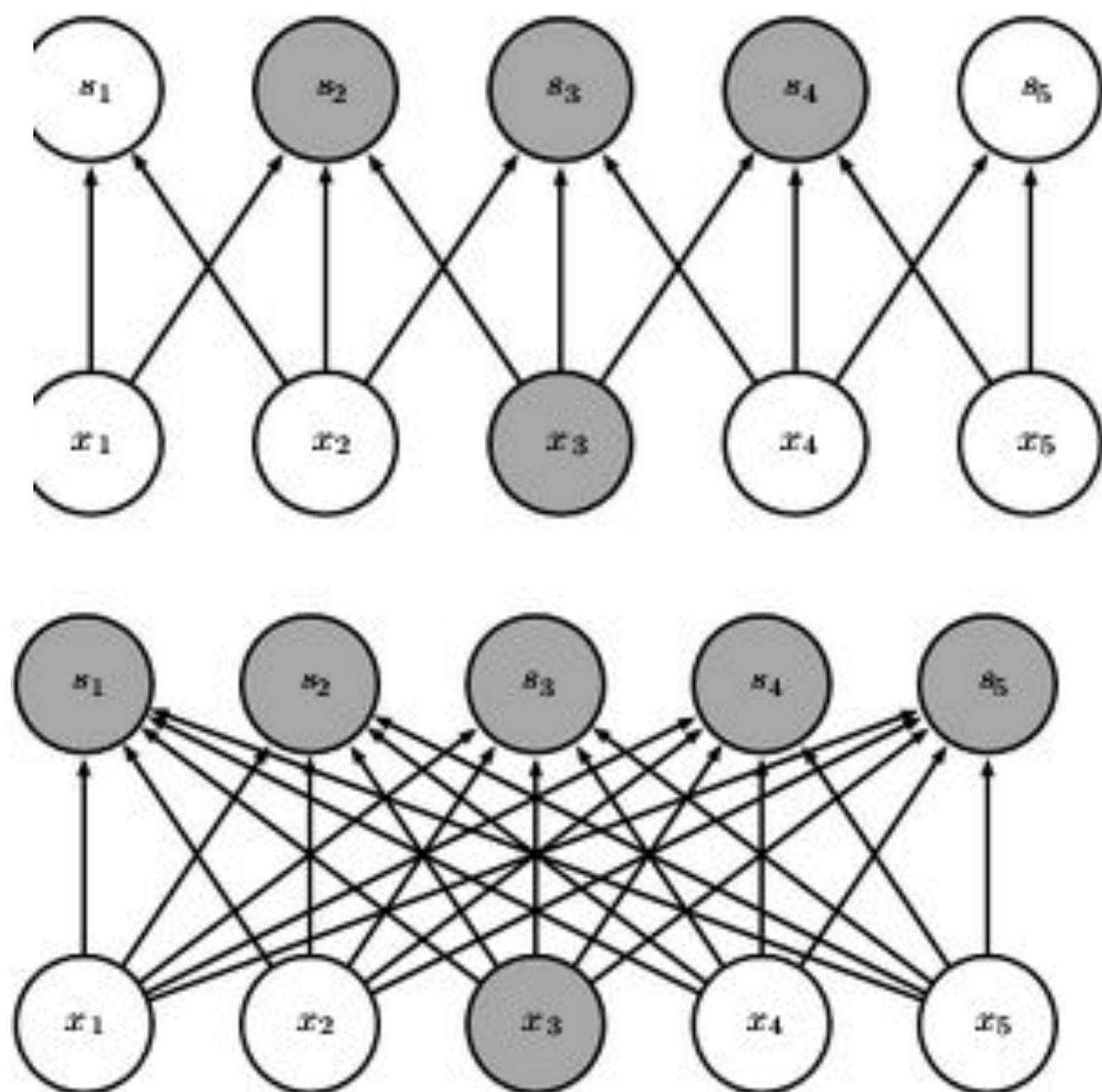


Figure 9.2: Sparse connectivity, viewed from below. We highlight one input unit, x_3 , and highlight the output units in s that are affected by this unit.

(Top) When s is formed by convolution with a kernel of width 3, only three outputs are affected by x .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all the outputs are affected by x_3 .

If there are m inputs and n outputs, then matrix multiplication requires $m \times n$ parameters, and the algorithms used in practice have $O(m \times n)$ runtime (per example). If we limit the number of connections each output may have to k , then the sparsely connected approach requires only $k \times n$ parameters and $O(k \times n)$ runtime.

For many practical applications, it is possible to obtain good performance on the machine learning task while keeping k several orders of magnitude smaller than m .

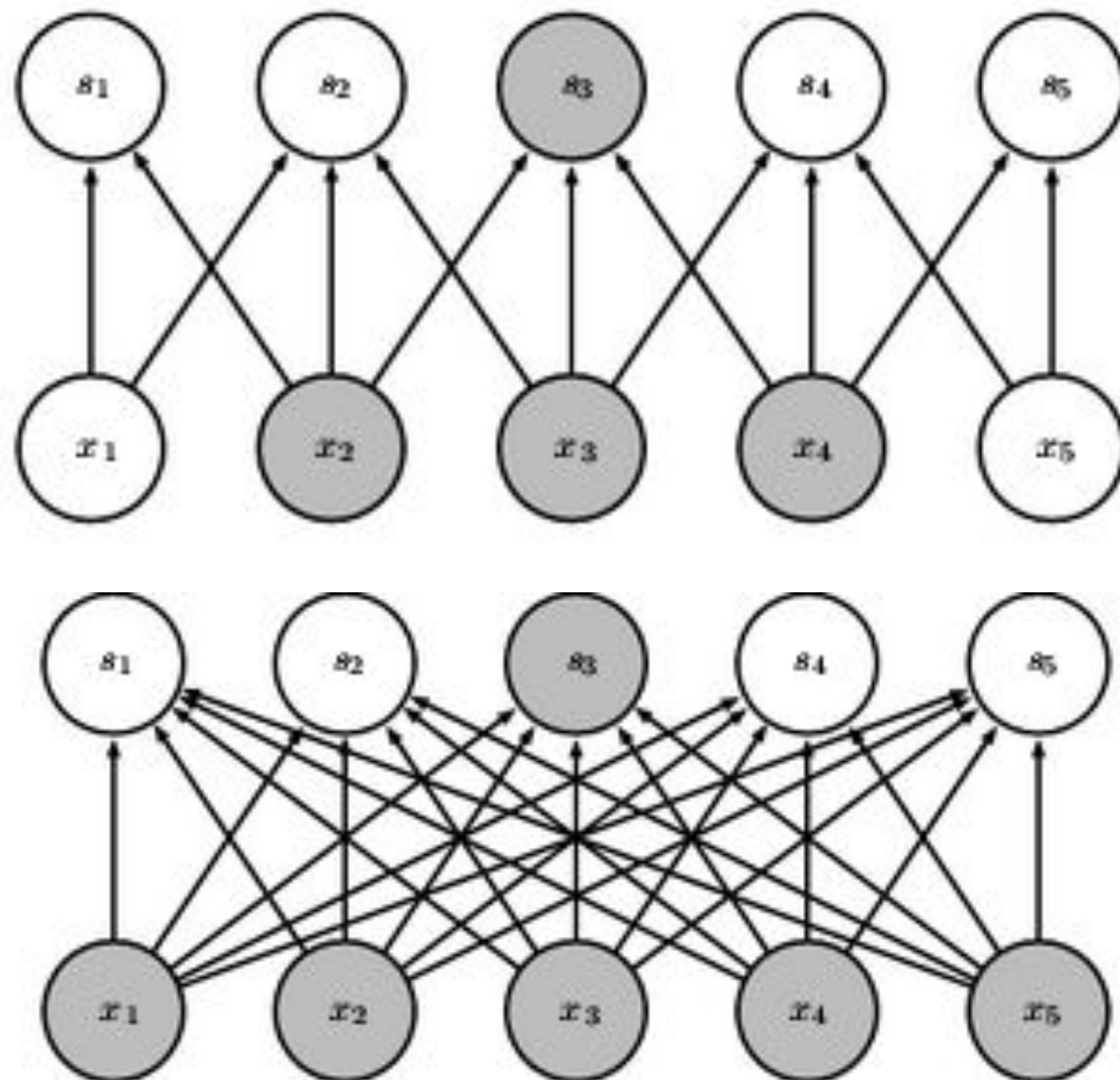


Figure 9.3: Sparse connectivity, viewed from above. We highlight one output unit, s_3 , and highlight the input units in x that affect this unit. These units are known as the receptive field of s_3 .

(Top) When s is formed by convolution with a kernel of width 3, only three inputs affect s_3 .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all the inputs affect s_3 .

In a deep convolutional network, units in the deeper layers may indirectly interact with a larger portion of the input, as shown in figure. This allows the network to efficiently describe complicated interactions between many variables by constructing such interactions from simple building blocks that each describe only sparse interactions

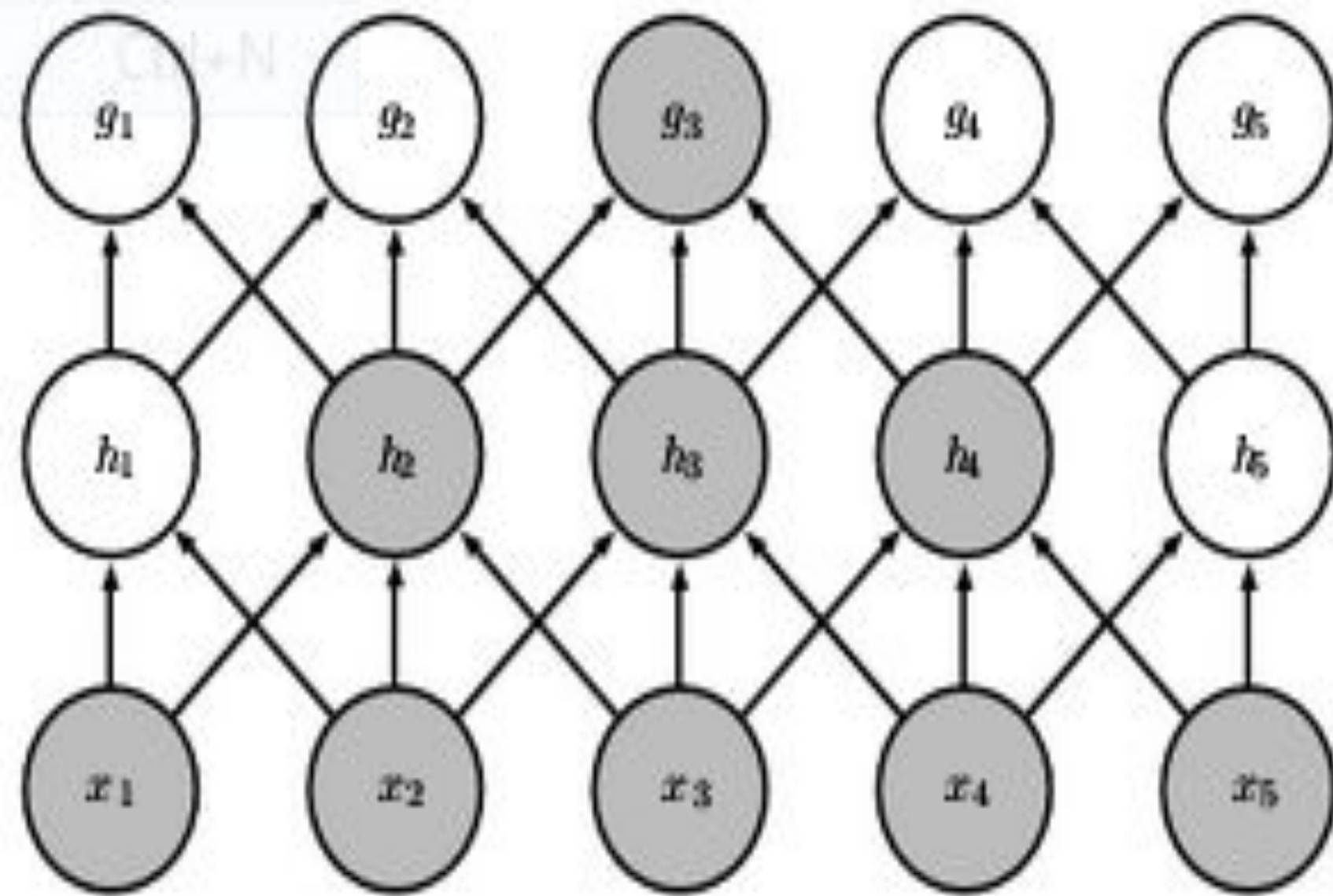


Figure 9.4: The receptive field of the units in the deeper layers of a convolutional network is larger than the receptive field of the units in the shallow layers. This effect increases if the network includes architectural features like strided convolution (figure 9.12) or pooling (section 9.3). This means that even though direct connections in a convolutional net are very sparse, units in the deeper layers can be indirectly connected to all or most of the input image.



Parameter sharing : using the same parameter for more than one function in a model



**RV College of
Engineering®**

Go, change the world

Thank You

